5 Contact-implicit Model Predictive Control

In the contact-implicit DDP framework, we integrate full-body robot dynamics with contact dynamics, solving the optimal control problem for the 3D case through Box-FDDP in Crocoddyl's solver (Mastalli et al. 2020), and for the 2D case through constrained DDP implemented in the DDP codebase (Tassa 2015). For the computation of the dynamics and its analytical derivatives, we use Pinocchio library (Carpentier et al. 2019; Carpentier and Mansard 2018).

This framework is applied for a quadruped robot that has a single contact point per foot, $d = 4$. We model each foot contact as a point contact, rather than a surface contact. In all experiments and 3D simulations using the HOUND quadruped robot, the optimal control problem is discretized with a time step of $dt = 25$ ms, has a horizon of $N = 20$, and runs in an MPC fashion at 40 Hz.

5.1 Cost

In this paper, four types of costs are utilized: regulating cost $l_r(\mathbf{x}_i, \mathbf{u}_i)$, foot slip and clearance cost $l_f(\mathbf{x}_i, \mathbf{u}_i)$, air time cost $l_a(\mathbf{x}_i)$, and symmetric control cost $l_s(\mathbf{u}_i)$. The regulating cost alone enables multi-contact motion; however, we introduce three additional costs to ensure tractable motion. For the i-th time step, the running cost $l(\mathbf{x}_i, \mathbf{u}_i)$ and the final cost $l_N(\mathbf{x}_N)$ are specified as follows:

$$\begin{aligned} l(\mathbf{x}_i, \mathbf{u}_i) &= l_r(\mathbf{x}_i, \mathbf{u}_i) + l_f(\mathbf{x}_i) + l_a(\mathbf{x}_i) + l_s(\mathbf{u}_i), \\ l_N(\mathbf{x}_N) &= l_{r,N}(\mathbf{x}_N), \end{aligned} \tag{20}$$

where $\mathbf{x}_i = (\mathbf{q}_i, \dot{\mathbf{q}}_i)$ is the robot state, $\mathbf{u}_i \in \mathbb{R}^{n_j}$ is the control input (joint torque), and $l_{r,N}(\mathbf{x}_N)$ is the regulating cost for terminal state. $l_r(\mathbf{x}_i, \mathbf{u}_i)$ and $l_f(\mathbf{x}_i, \mathbf{u}_i)$ are applied in all 3D simulations and experiments, while $l_a(\mathbf{x}_i)$ and $l_s(\mathbf{u}_i)$ are specifically used for encouraging symmetric gaits.

5.1.1 Regulating Cost. The regulating cost is a quadratic form designed to minimize the error between the desired and actual values for both configuration and control input:

$$\begin{aligned} l_r(\mathbf{x}_i, \mathbf{u}_i) &= \|\epsilon_{\mathbf{x}_i}\|^2_{\mathbf{W}_\mathbf{x}} + \|\mathbf{u}_i - \mathbf{u}_{i,\text{ref}}\|^2_{\mathbf{W}_\mathbf{u}}, \\ l_{r,N}(\mathbf{x}_N) &= \|\epsilon_{\mathbf{x}_N}\|^2_{\mathbf{W}_{\mathbf{x}_N}}, \end{aligned} \tag{21}$$

where $\epsilon_{\mathbf{x}_i} := \mathbf{x}_i \ominus \mathbf{x}_{i,\text{ref}}$ incorporates the SO(3) error, where the body rotational error is defined as $\epsilon_{\mathbf{R}_i} := \log(\mathbf{R}^T_{i,\text{ref}}\mathbf{R}_i)^\vee$ using the logarithm map and the vee operator.[1] Errors in translation, joint configuration, and generalized velocity, are the differences between the desired and actual state. The corresponding diagonal weight matrices $\mathbf{W}_\mathbf{x}$, $\mathbf{W}_\mathbf{u}$, and $\mathbf{W}_{\mathbf{x}_N}$ are for states, control inputs, and the terminal state, respectively.

5.1.2 Reference Trajectory. In all experiments and 3D simulations with the quadruped robot, we solely set the reference body's position and orientation in SE(3). All the other components are left at zero or in their fixed nominal configurations. Specifically, both $\dot{\mathbf{q}}_{i,\text{ref}}$ and $\mathbf{u}_{i,\text{ref}}$ are set to zero. The joint component of $\mathbf{q}_{i,\text{ref}}$ remains fixed as the nominal stance configuration; leg movements are regulated based on this nominal configuration, implying no pre-planned references for leg trajectories or foothold positions.

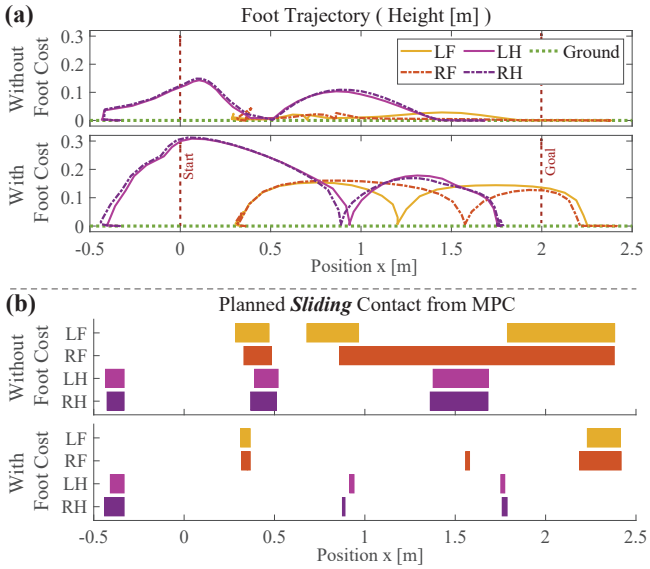

Figure 5. The effect of foot slip and clearance cost is depicted. Two scenarios are considered: "With Foot Cost", incorporating both regulating cost and foot slip and clearance cost, and "Without Foot Cost", using only the regulating cost. For both cases, the target position is set 2.0 m ahead of the initial position. The resultant swing leg trajectory is depicted in (a), and the planned sliding contact is illustrated in (b).

We maintain the consistent $\mathbf{x}_{i,\text{ref}}$ without interpolating across all $i \in \{0, 1, \cdots, N\}$, highlighting that fixed, elementary reference trajectories suffice for desired motions.

5.1.3 Foot Slip and Clearance Cost. In the absence of pre-planned references for leg trajectories or foothold positions, solely using the regulating cost does not improve foot lifting or prevent slipping motion. This can result in minimal foot-lifting (Posa et al. 2014) or prevalent scuffing (Carius et al. 2019), leading to challenges in real-world tracking and potential failures. Similar to Shirai et al. (2022), we add additional cost for these issues. Inspired by the foot slip cost and foot clearance cost commonly used in reinforcement learning framework (Hwangbo et al. 2019; Ji et al. 2022), we formulate the individual costs as a single differentiable function for foot slip and clearance cost:

$$l_f(\mathbf{x}_i) = c_f \sum_{k=1}^{4} S(c_1\phi_{k,i})||\mathbf{v}^t_{k,i}||^2, \tag{22}$$

where $S(c_1\phi_{k,i}) = 1/(1 + e^{(-c_1\phi_{k,i})})$ is the Sigmoid function to map the foot height as a scalar weight. $\phi_{k,i}$ and $\mathbf{v}^t_{k,i}$ denote the foot height and the tangential foot velocity, respectively. c_1 tunes the steepness of the Sigmoid function (set to -30), and c_f is the total weight of the cost. Here, $S(c_1\phi_{k,i})$ approaches a value of 0.5 for small foot heights ($\phi_{k,i} \approx 0$), and approaches zero as the foot height $\phi_{k,i}$ increases. As a result, the tangential foot velocity is penalized when $\phi_{k,i} \approx 0$ (preventing foot slip), and the foot is encouraged to be high when the foot has tangential velocity (ensuring foot clearance), as shown in Figure 5.

5.1.4 Air Time Cost. Considering various contact points, there could be multiple contact modes to track a reference configuration, allowing a quadruped to reach its target position using fewer than four legs, such as walking with

only three legs. Specific gaits, such as trotting, often require fine-tuning of weights to balance body motion, joint motion, and control input (Neunert et al. 2017). In our work, we introduce an air time cost that penalizes prolonged swing leg time during walking, without the need for highly fine-tuned parameters for specific gaits:

$$l_a(\mathbf{x}_i) = \sum_{k=1}^{4} c_a[k,i]\phi_{k,i}^2, \tag{23}$$

where $c_a[k,i]$ denotes the scalar weight of the air time cost for the k-th contact point at time step i. All $c_a[k,i]$ values are initialized to 0, and are activated as a positive weight c_a only when swing leg time surpasses threshold $i_t = 12$ (0.3 sec). From the initialized trajectory, we identify legs with excessive swing time over the entire horizon. For those legs, $c_a[k,i]$ is set to c_a for time steps i_t to $i_t + 3$ (during four time steps), penalizing foot lift with $\phi_{k,i}^2$. Once activated, this weight propagates in subsequent MPC problems until it reaches the beginning of the horizon (time step 0).

5.1.5 Symmetric Control Cost. The symmetric control cost aims to facilitate symmetric gait motions such as trotting and pacing. Here, the symmetry is emphasized within paired legs.[2] The cost penalizes the torque differences for the hip flexion/extension (HFE) and knee flexion/extension (KFE) of the paired legs:

$$l_s(\mathbf{u}_i) = c_s||\mathbf{C}_2\mathbf{u}_i||^2, \tag{24}$$

where the c_s denotes the scalar weight and $\mathbf{C}_2 \in \mathbb{R}^{4\times n_j}$ (with $n_j = 12$) is a constant matrix which maps the joint torques to the torque differences between the paired legs. In trot motion, diagonal legs are paired, while in pacing motion, same-side legs are paired. For example, in the case of diagonal pair, $\mathbf{C}_2^{\text{diagonal}}$ is:

$$\mathbf{C}_2^{\text{diagonal}} = \begin{bmatrix} \mathbf{D} & \mathbf{0}_{2\times3} & \mathbf{0}_{2\times3} & -\mathbf{D} \\ \mathbf{0}_{2\times3} & \mathbf{D} & -\mathbf{D} & \mathbf{0}_{2\times3} \end{bmatrix},$$

where the matrix $\mathbf{D}$ is defined as $\mathbf{D} = [\mathbf{0}_{2\times1}, \mathbf{I}_{2\times2}]$. Importantly, this cost function is not time-dependent, indicating that it is not activated periodically to induce a specific gait motion.

5.2 Roll-out for the Forward Pass

In the forward pass, the occurrence of contact is determined by evaluating the foot height condition, $\phi_{k,i} < 0$ or $\phi_{k,i+1} < 0\ \forall k \in \{1, \cdots, 4\}$, based on the current state $\mathbf{x}_i$ and control $\mathbf{u}_i$. By approximating the next foot height using a first-order Taylor expansion, this condition ($\phi_{k,i+1} < 0$) is represented as a velocity space condition with a drift term, thus leading to an affine condition for the contact impulse:

$$\begin{aligned} \phi_{k,i+1} &= \text{FK}_k(\mathbf{q}_{i+1}) = \text{FK}_k(\mathbf{q}_i + \dot{\mathbf{q}}_{i+1}dt) \\ &\approx \mathbf{J}_k^n(\mathbf{q}_i)\dot{\mathbf{q}}_{i+1}dt + \text{FK}_k(\mathbf{q}_i) = v_{k,i+1}^n dt + \phi_{k,i}, \end{aligned} \tag{25}$$

where FK_k denotes the forward kinematics of the k-th contact point. Then, for the identified contact case, we compute the contact impulse based on the strict complementarity constraint, as detailed in Section 3.3. For preventing the occurrence of penetration due to the large dt, we enforce $v_{k,i+1}^n + \frac{\phi_{k,i}}{dt} = 0$ instead of the original velocity space constraint $v_{k,i+1}^n = 0$. The details of this drift compensation are described in Appendix B. Subsequently, the resultant contact impulse is used to integrate the state of the next time step via the semi-implicit Euler method.

5.3 Gradient for Backward Pass

In the backward pass, we compute the analytic gradient of the contact impulse based on the relaxed complementarity constraint. Then, the computed gradients of contact impulse are used in obtaining the gradient $\mathbf{f_x}$ and $\mathbf{f_u}$ using the analytical derivative of the articulated body algorithm (ABA) (Carpentier and Mansard 2018; Carpentier et al. 2019). Notably, the entire gradient computation takes around 70 μs under the most demanding scenario with all four contact points in clamping cases, tested on a desktop PC with an AMD Ryzen 5 3600X processor. The details of the computation are in Appendix B.

6 Implementation Detail for Motion Execution

The optimization process is initialized by utilizing the state-control trajectory $\mathbf{x}_0, \ldots, \mathbf{x}_{N-1}$, $\mathbf{u}_0, \ldots, \mathbf{u}_{N-2}$, which is shifted by one time step from the solution of the previous MPC problem.[3] This initialization provides an effective starting point for subsequent optimization problem (Diehl et al. 2002). It enhances convergence and guides the optimization towards the intended local minimum, ensuring motion continuity.

However, the *actual* initial state, represented by $\tilde{\mathbf{x}}_0$ (the current estimated state), often deviates from $\mathbf{x}_0$, the state anticipated in the previous MPC problem. This discrepancy leads to complications within contact-implicit approach. In single shooting schemes, initialization is performed by successively integrating the control input trajectory, $\mathbf{u}_0, \ldots, \mathbf{u}_{N-1}$, starting from the *deviated* initial state, $\tilde{\mathbf{x}}_0$, rather than $\mathbf{x}_0$. Given the hybrid model inherent in the contact-implicit method, even slight discrepancies can prompt a cascade of contact mode changes throughout the prediction horizon (Posa et al. 2014). This frequently results in trajectory divergence, leading to motion failures.

To address the aforementioned challenges, the FDDP algorithm (Mastalli et al. 2020, 2022a), a multiple-shooting variant of DDP, is employed, as multiple shooting (Bock and Plitt 1984) could reduce the high sensitivity issue of single shooting (Wensing et al. 2023). FDDP accepts state-control trajectories for initialization, where the initialized state trajectory, $\mathbf{x}_0, \ldots, \mathbf{x}_N$, implicitly guides contact modes consistent with the previous solution. Therefore, the initial state discrepancy ($\tilde{\mathbf{x}}_0 \neq \mathbf{x}_0$) does not lead to divergence of trajectory due to unwanted contact mode variations such as in single shooting scheme. Although the dynamics gap exists due to initial state discrepancy, the FDDP algorithm progressively reduces these gap in the early stage of optimization.

In the following subsections, we detail this issue by contrasting the single shooting and multiple shooting schemes in the contact-implicit MPC framework. Additionally, an overview of the feedback control is provided.

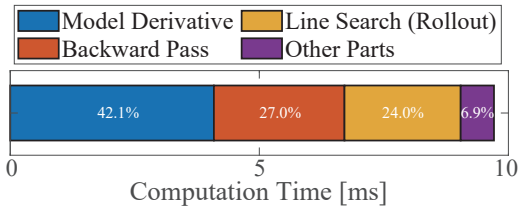

Figure 20. Time profiling for solving an MPC problem.

recursively computes the optimal control step. The third component is the line search process, including the forward roll-out, primarily due to the contact impulse computations.

Weight Setting

We have created a Table 2 to denote the weights for 3D quadruped simulation and experiment, for the regulating cost l_r, foot slip and clearance cost l_f with weight c_f, air time cost l_a with weight c_a, and symmetric control cost l_s with weight c_s. The regulating cost weight matrix for state, $\mathbf{W_x}$, is derived by adjusting the positional component, $\mathbf{W_q}$, and the velocity component, $\mathbf{W_{\dot{q}}}$, with $\mathbf{W_{\dot{q}}} = \mathbf{W_q}/\alpha$, where α is the velocity weight factor. The final state weight matrix, $\mathbf{W_{x_N}}$, is calculated as $\beta\mathbf{W_x}$, with β representing the final weight factor. The regulating cost weight matrix for control input $\mathbf{W_u}$ is tuned using the diagonal element $c_\mathbf{u}$.

For all tasks not related to walking (which includes trotting and pacing), the air time cost weight and symmetric control weight are set to zero: $c_a = 0, c_s = 0$. For fine-tuning in simulations, we change the weights as follows: $c_\mathbf{u} = 1 \times 10^{-4}$ for the target roll of 0.6 rad, and $\alpha = 1 \times 10^2, c_s = 1 \times 10^{-3}$ for the target height of +0.25m. In the trotting experiment, we set $\alpha = 3.5 \times 10^1$ and $c_\mathbf{u} = 3 \times 10^{-4}$. The snapshots of the resulting motions and the detailed cost types for Extensions 4 and 5 are described in Figure 23. In the comparison with the relaxed forward model in Section 7.2.7, we set the weights for all body parts and joint parts of l_r to 1×10^1 and 1×10^0, respectively, and $\alpha = 2 \times 10^2$, $c_\mathbf{u} = 1 \times 10^{-5}$ for both methods.

Table 2. Default weight setting for 3D quadruped simulation and experiment

Cost	Parameter	Value
l_r	Body position x,y	2×10^1
	Body position z	8×10^1
	Body rotation	1×10^1
	Joint position	1×10^0
	Control input ($c_\mathbf{u}$)	2×10^{-4}
	Velocity weight factor (α)	3×10^1
	Final weight factor (β)	1×10^1
l_f	Weight (c_f)	1×10^0
l_a	Weight (c_a)	2×10^3
l_s	Weight (c_s)	1×10^{-2}
	Relaxation Variable (ρ)	2×10^0

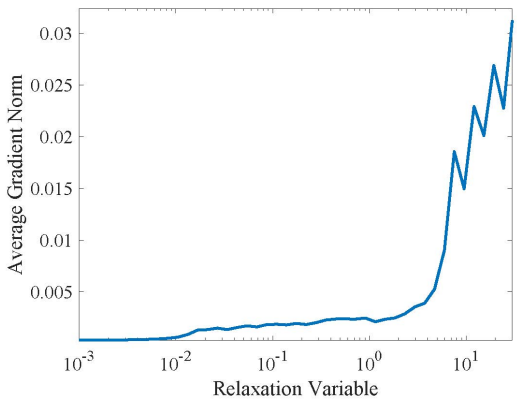

Figure 21. Average gradient norm at convergence points along relaxation variable for the 2D quadruped jumping task.

Appendix C: Further Discussion

Convergence Issue with Excessive Relaxation Values.

Excessively large relaxation values can cause the relaxed gradient to deviate from the descent direction, potentially degrading convergence. In such cases, the line search process prevents cost increases by halting the step size. Figure 21 compares the norm of the exact gradient at convergence points for each relaxation variable within the same 2D simulation jumping task as in Figure 8. The norm of the exact gradient (y-axis of Figure 21) indicates how far the convergence point is from the local minima.

As the relaxation variable increases, the gradient norm progressively rises, positioning convergence points further from the local minima and potentially impeding convergence due to deviations from the descent direction. Notably, there are two abrupt jumps in the gradient norm (Figure 21), corresponding with significant cost changes (Figure 8). When the relaxation variable is very small (less than 0.02), it fails to find the jumping motion. As the relaxation variable increases, it discovers jumping motions, leading to better optimal points (reduced cost) with only a slight increase in the gradient norm; this range of the relaxation is what we intend to utilize. However, values exceeding 2.0 significantly increase both cost and gradient norm; convergence to local minima is not achieved, indicating that the line search fails to reduce the cost.

Dynamics Gap of Front-Leg Rearing Motion.

Figure 22 shows the dynamics gap over time for the front-leg rearing experiment in Section 7.3.3 for all 6 trials. This figure illustrates instances where the motion is executed without completely closing all dynamics gaps. This is primarily due to the difficulty of the tasks.

Front-leg rearing involves highly dynamic motions with only one or two feet making brief contacts, each critical for the continuation of the motion. Rapid changes in contact and the need for large ground reaction forces to support the robot's body weight mean that slight contact mismatches can lead to significant deviations, particularly in joint velocity. Despite these challenges, the subsequent MPC iterations close the gaps over time and stabilize the robot.